

RUDP: A Reliable Transport Protocol Built on UDP

Design, implementation, and validation of a custom transport-layer protocol replicating TCP's core reliability and congestion-control mechanisms — built from raw sockets, tested against real network emulation.

Harshavardhan Mallela · 24BAI0274 Rohit Arya · 24BCE0819 Shloak Sinha · 24BDS0378

ABSTRACT

The Transmission Control Protocol (TCP) provides reliable, ordered, congestion-aware delivery over an unreliable network layer, but its internal mechanisms are often studied only theoretically. This project bridges that gap by designing and implementing **RUDP (Reliable UDP)**, a custom transport-layer protocol built entirely on UDP sockets that replicates the core reliability and congestion-control mechanisms found in TCP.

The protocol implements connection establishment via a three-way handshake, reliable in-order delivery using sequence numbers and cumulative acknowledgments, retransmission on timeout and triple-duplicate ACKs, sliding-window flow control, and AIMD-based congestion control with slow start and congestion avoidance. Correctness was validated under real OS-level packet loss using Linux `tc/netem`, with traffic independently verified via Wireshark packet capture, and performance benchmarked across loss conditions from 0% to 40%.

IMPLEMENTATION TIMELINE

PHASE 0

Environment & toolchain

WSL2 + Ubuntu, gcc/valgrind/tc/git installed, GitHub repository initialized, build system verified.

PHASE 1

Raw UDP echo

Baseline client/server communicating over a UDP socket — confirmed the transport layer works before any reliability logic was added.

PHASE 2

Packet format & sequencing

Custom 13-byte header — sequence number, ack number, flags, checksum, payload length — with manual byte-level pack/unpack to avoid struct padding issues across platforms. Verified with round-trip and corruption-detection tests, Valgrind-clean.

PHASE 3

Connection establishment

Three-way handshake (SYN → SYN-ACK → ACK) with a connection state machine (LISTEN → SYN_RECEIVED → ESTABLISHED), plus FIN-based teardown.

PHASE 4

Reliable delivery

Go-Back-N sliding window with cumulative ACKs, retransmission timeout, and fast retransmit on three duplicate ACKs. Verified under 40% simulated loss with byte-perfect message reassembly.

PHASE 5

Congestion control

AIMD — slow start doubling the window per RTT, congestion avoidance growing it linearly past `ssthresh`, and asymmetric loss response: a full reset to `cwnd=1` on timeout, a gentler halving on fast retransmit.

PHASE 6

Network emulation & capture

Real OS-level loss injection via `tc/netem` on the loopback interface, independently verified with Wireshark/tshark packet captures — confirming the retransmission timer fires at the expected interval on real network degradation, not just simulated conditions.

PHASE 7

Benchmarking & visualization

Congestion window logged to CSV across every run, plotted to reveal the AIMD sawtooth. A benchmark sweep across seven loss levels quantified how transfer duration and final window size scale with network conditions.

RESULTS

Across every tested loss level — 0% through 40% — all 62 packets of the test transfer were delivered with zero data loss and zero corruption. The two figures below characterize how the protocol behaves under stress.

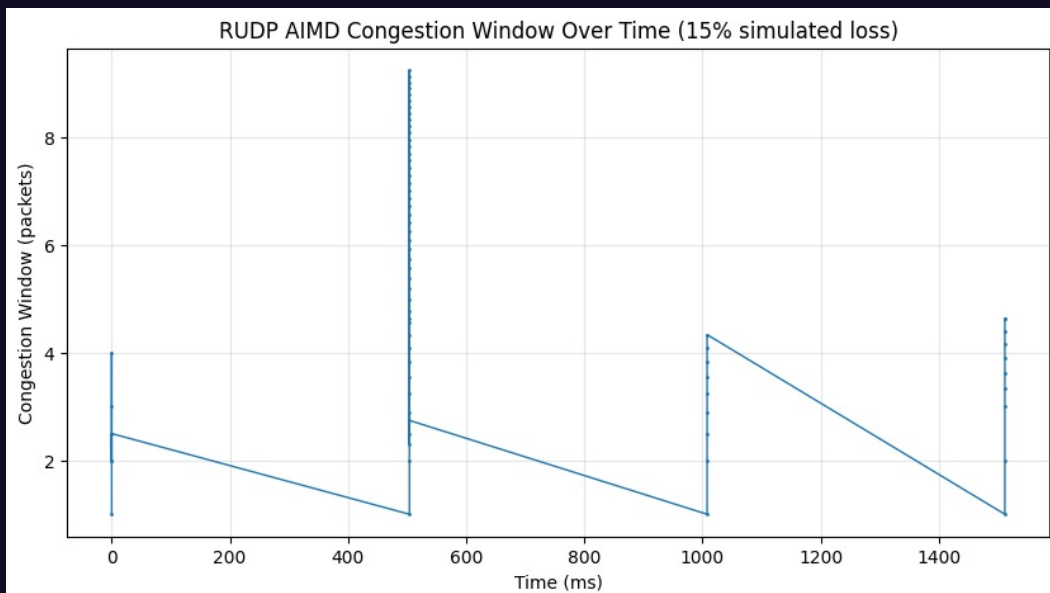


FIG. 1 — Congestion window over time under 15% simulated loss. Growth (slow start / congestion avoidance) is repeatedly interrupted by loss events, producing the characteristic AIMD sawtooth.

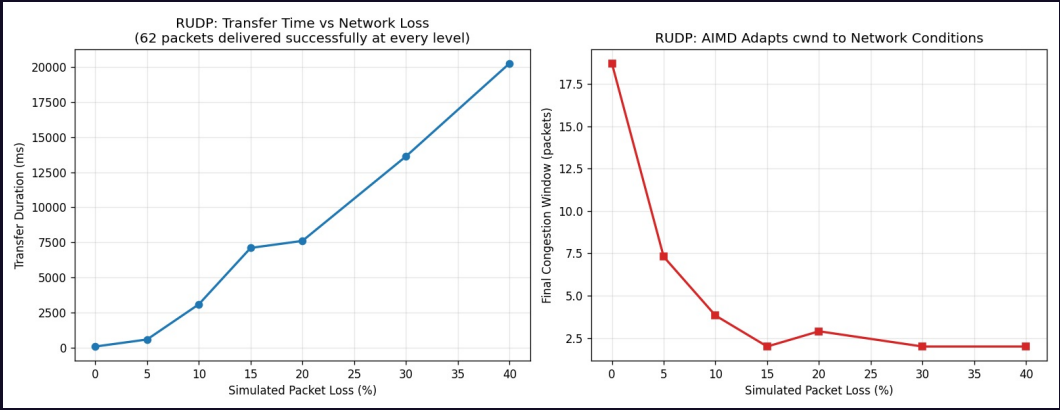


FIG. 2 — Left: transfer duration scales with loss rate as retransmissions accumulate. Right: AIMD correctly shrinks the final congestion window as loss increases, trading throughput for stability rather than continuing to hammer a degraded network.

Loss %	Duration	Packets delivered	Final cwnd
0%	108 ms	62 / 62	18.71
5%	608 ms	62 / 62	7.30
10%	3,112 ms	62 / 62	3.83
15%	7,118 ms	62 / 62	2.00
20%	7,617 ms	62 / 62	2.90
30%	13,625 ms	62 / 62	2.00
40%	20,236 ms	62 / 62	2.00

100%

0

delivery rate at every loss level tested

corrupted or missing bytes across all runs

MISTAKES MADE & HOW THEY WERE FIXED

Kept deliberately — these were real engineering obstacles, not smooth sailing, and diagnosing them is as much a part of understanding transport protocols as the design itself.

CONNECTION STATE BUG

Server silently rejected every new connection after one crashed client

The server only accepted a fresh SYN while in the `LISTEN` state. If a client was killed mid-session (Ctrl+C, a script timeout), the server was left stuck in `SYN_RECEIVED` or `ESTABLISHED` and ignored all future handshake attempts — surfaced as a benchmark sweep where every single loss level failed identically, taking exactly 15.5 seconds (five handshake retries × three-second timeout) with zero packets ever delivered.

FIX The server now accepts a new SYN in any connection state, not just `LISTEN` — matching how a real server must tolerate an abandoned or crashed peer without needing a manual restart.

NETWORK TESTING

A leftover netem rule silently stacked with the next test

After a Wireshark capture session using `tc qdisc add dev lo root netem`

`loss 20%` , the rule was never removed. The next benchmark run then combined that 20% with its own simulated loss argument, multiplying effective loss far beyond what was intended and making an otherwise-correct run look like a hang.

FIX Always verify with `tc qdisc show dev lo` before trusting a result, and explicitly remove the rule with `tc qdisc del dev lo root netem` immediately after any netem-based test.

TOOLING

Wireshark's Windows GUI couldn't see WSL2's internal loopback traffic

Capturing on the `vEthernet (WSL)` interface from native Windows Wireshark showed 0 matching packets — traffic between two processes both running inside WSL2 over `127.0.0.1` never actually crosses out to the Windows-visible virtual adapter.

FIX Captured directly inside WSL using `tshark -i lo` , writing to a `.pcapng` file, then copied that file to Windows to inspect in the full Wireshark GUI.

SHELL PORTABILITY

Benchmark timing and extraction logic silently produced garbage

A millisecond-timing command (`date +%s%3N`) returned malformed output on this system, producing a nonsensical multi-day "duration." Separately, `grep -oP` (PCRE-mode regex) failed to extract packet counts and final window size, leaving those fields blank in every result row.

FIX Switched to `date +%s.%N` with an `awk` -based millisecond calculation, and replaced PCRE extraction with plain, portable `sed` / `grep` patterns.

ENVIRONMENT

Assumed filesystem paths that didn't match reality

Windows and WSL usernames are entirely independent — copying capture files to an assumed `/mnt/c/Users/Harsh/Downloads/` failed with "No such file or directory" because the actual Windows account was named differently.

FIX `ls /mnt/c/Users/` to confirm the real username before referencing any Windows-side path from WSL.

SECURITY HYGIENE

A GitHub access token was exposed in a screenshot

A Personal Access Token was visible in a shared screenshot during setup — any credential that appears in a screenshot or chat log should be treated as compromised, regardless of the channel.

FIX Revoked the exposed token immediately on GitHub and generated a fresh one, entering it directly into the terminal prompt from then on rather than displaying it anywhere.

CONCLUSION

RUDP demonstrates that TCP's core reliability guarantees — ordered, complete delivery over a fundamentally unordered, lossy transport — can be built and verified from first principles. Every mechanism was validated independently: packet-level correctness through unit tests, reliability through induced loss up to 40%, and congestion behavior through both live logging and real OS-level network emulation cross-checked against Wireshark ground truth.

The most significant limitation is the choice of Go-Back-N over Selective Repeat: a single lost packet forces retransmission of the entire in-flight window rather than just the missing piece, which is simpler to reason about but less bandwidth-efficient under loss. A natural extension is a Selective-Repeat receiver with per-packet ACKs,

alongside a second congestion-control variant (a Cubic-style window function) to compare directly against the AIMD results already gathered here.

github.com/Capsturrrr/rudp — VIT Vellore, Computer Networks